

and becomes “pixelated.” An interface built using such pixel-based elements, or bitmaps, is termed raster-based.

Being pixel-oriented, the GDI was also a bottleneck to readability as resolutions of monitors increased. A pixel-based system defines elements in absolute measures of pixels: you increase the resolution and the pixels get smaller, reducing the size of the element drawn as well. So while the GDI system suited the older display hardware well, as displays with higher pixel density became more common, you needed to squint at the screen to read anything. A raster API such as the GDI also had problems when it came to scaling and animating windows (transforms). Due to the rudimentary nature of earlier video cards, most of these transforms were performed by the CPU. Old video cards barely had enough memory to remember little more than the final screen that was to be rendered on your monitor. Moreover, they lacked the processing power to do any mathematical calculations on their own. Since GDI was programmed amidst this hardware status quo, the API lagged innovations in hardware.

The cards were then little more than a holding space for framebuffers (a framebuffer is memory that stores the video frame that needs to be displayed on screen). The cards would just be able to store that final image. It was the processor that needed to do the tasks of actually manipulating images to make them fit the myriad boxes, toolbars, menus, and so on. Being nothing more than a framebuffer also meant that the cards—and therefore the API—that leveraged the card did not keep historical track of various on screen elements. Say, for example, a desktop with two windows was rendered in this scenario: the window on top was an Explorer window browsing the C:, while just behind it was a Web browser window displaying a page.

Picture the Explorer window overlapping some area of the browser behind it. In this scenario, the API and the video card would just render the final scene of the two overlapping, without storing any information about the area of the browser window